

釣行AIアプリ AWS設計統合書（実務版 v2）

本書は、釣行AIアプリの「要件定義・AWS基本設計・Lambda設計・AI設計」を統合し、実装の唯一の基準とする。

第1章：システム概要（要件定義）

1. システム目的

- 釣り場検索ではなく「釣行意思決定AI」
- ユーザーが最短で「どこで何を釣るか」を決定できる
- PWA（スマホ・PC対応）
- 個人～少人数利用前提

2. コア機能

- TOP3スポット推薦
- スコアリング（AI+ルール）
- 地図表示（Google Maps）
- 釣果データ表示
- お気に入り管理

3. スコアロジック

```
score =  
  fish_probability * 0.4  
+ weather_score * 0.2  
+ tide_score * 0.2  
- distance_penalty * 0.1  
- cost_penalty * 0.1
```

第2章：システムアーキテクチャ（AWS基本設計）

ユーザー（PWA）
↓
CloudFront
↓
S3（Frontend）
↓
API Gateway
↓
Lambda（API / Batch）
↓
DynamoDB

コンポーネント役割

- CloudFront：配信高速化
- S3：Reactホスティング
- API Gateway：API管理
- Lambda：処理実行
- DynamoDB：データ管理

バッチ処理

EventBridge
↓
Lambda（データ収集）
↓
天気・潮汐・釣果API
↓
DynamoDB保存
↓
AIスコア生成
↓
Recommendations更新

リアルタイム方針

リアルタイム処理は禁止し、バッチ中心で設計する

- コスト削減

- レスpons安定化
- 外部依存排除

第3章：AI設計（重要）

1. 採用モデル

用途	モデル	理由
スコア生成	OpenAI GPT-4.1 / GPT-4o	数値処理とJSON出力安定
理由生成	Claude / GPT-4o	自然言語品質
コスト最適化	Gemini（補助）	低コストバッチ用途

2. AI設計方針

- リアルタイム対話は行わない
- バッチ or API呼び出しでスコア生成
- 結果はDynamoDBにキャッシュ保存
- AIは“意思決定エンジン”として扱う

3. 出力フォーマット

```
{
  "spotId": "A123",
  "score": 87,
  "fishTypes": ["アジ"],
  "reason": "潮流が安定しているため",
  "confidence": 0.82
}
```

第4章：Lambda詳細設計

1. 方針

- 1関数 = 1責務
- API系とバッチ系を分離
- DynamoDB経由アクセス

2. API Lambda

関数	用途
getRecommendations	TOP3取得
getSpots	スポット一覧
getPosts	釣果取得
createFavorite	お気に入り登録

3. バッチLambda

関数	用途
fetchWeatherBatch	天気取得
fetchTideBatch	潮汐取得
generateScoreBatch	AIスコア生成
updateRecommendationBatch	推薦更新

4. getRecommendations処理

1. Recommendations取得
2. 距離補正
3. スコア順ソート
4. TOP3抽出
5. JSON返却

第5章：データ設計（概要）

Spots

spotId (PK)
name
lat
lng

Recommendations

spotId (PK)
score
fishTypes
reason
distance
cost
weatherScore
tideScore

Favorites

userId (PK)
spotId (SK)
memo

第6章：非機能要件

- API応答：1～2秒以内
- Lambda自動スケール
- DynamoDBオンデマンド
- 99.9%可用性

第7章：システム思想

本システムは「検索アプリ」ではなく AIによる“釣行意思決定エンジン”である

第8章：MVP構成

- React + TypeScript
- AWS Lambda
- DynamoDB
- Google Maps
- AIスコアリング

第9章：まとめ

AIはフロントに出さず、裏側の意思決定エンジンとして統合することで、高速・低コスト・高精度な釣行判断システムを実現する。